

Homework 3

DATA 260

Name: Kazi Samina Maraj Mumu

SJSU ID: 018324098

Email: Kazisaminamaraj.mumu@sjsu.edu

Date: 09/17/2026

Introduction

In this homework, I continued my municipal transit incident application. I added a login system so that the dashboard is protected. I also compared three different ways of splitting transit documents into smaller pieces for a retrieval system. The goal was to see which method helped the system find useful information for a question.

My project uses the repository data260-4098, port 8498, and the municipal transit incident domain. I kept the work for this homework separate from my Lab 1 repository.

1 Authentication

I added a public home page, a login page, a protected dashboard, and a logout option. Anyone can see the home page and incident records, but the dashboard requires a valid login. If a person tries to open the dashboard without logging in, the application sends them to the login page.

The demo account I used was samina with the password Transit4098!. The application stores the login in a session cookie. The cookie is marked secure, HTTP-only, and SameSite=Lax. These settings help prevent the cookie from being exposed or used in unsafe requests.

The session also has an idle timeout. If the session is no longer active, the user must log in again.

Authentication screenshots

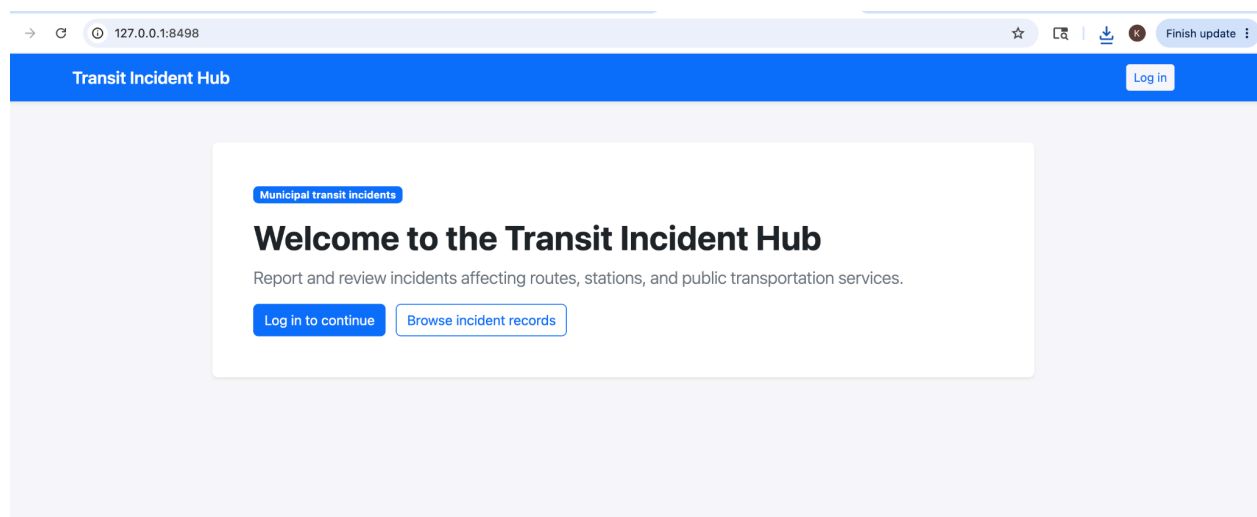


Figure 1. Public home page of the Transit Incident Hub.

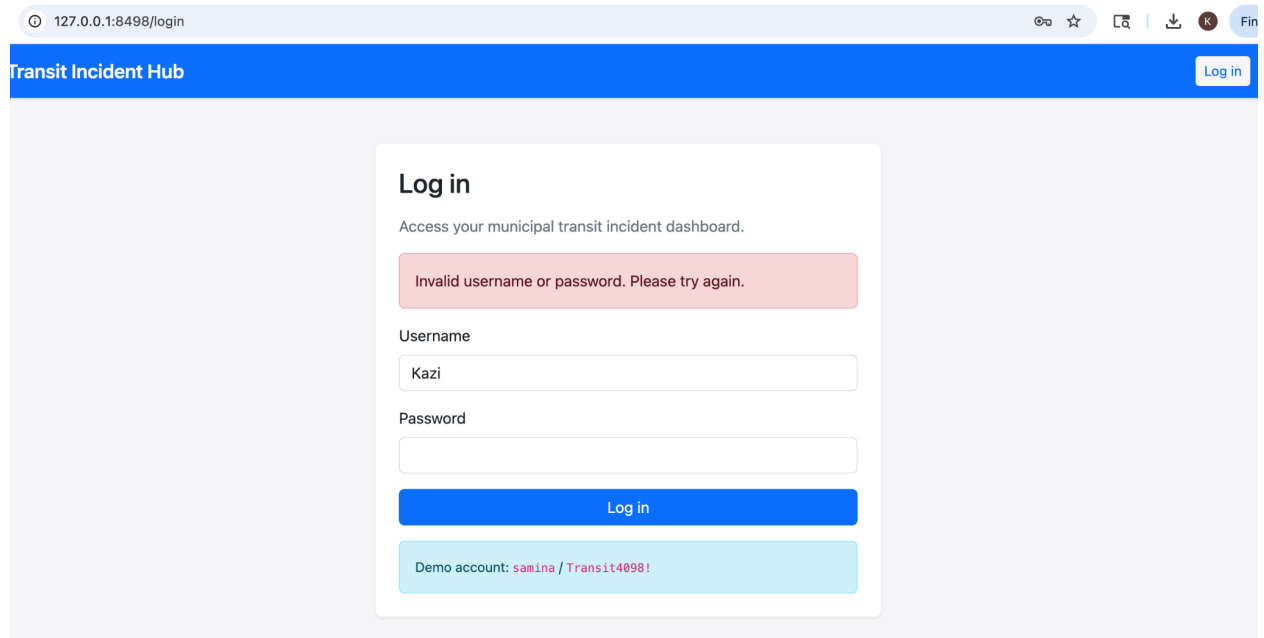


Figure 2. Login page showing an invalid username or password message.

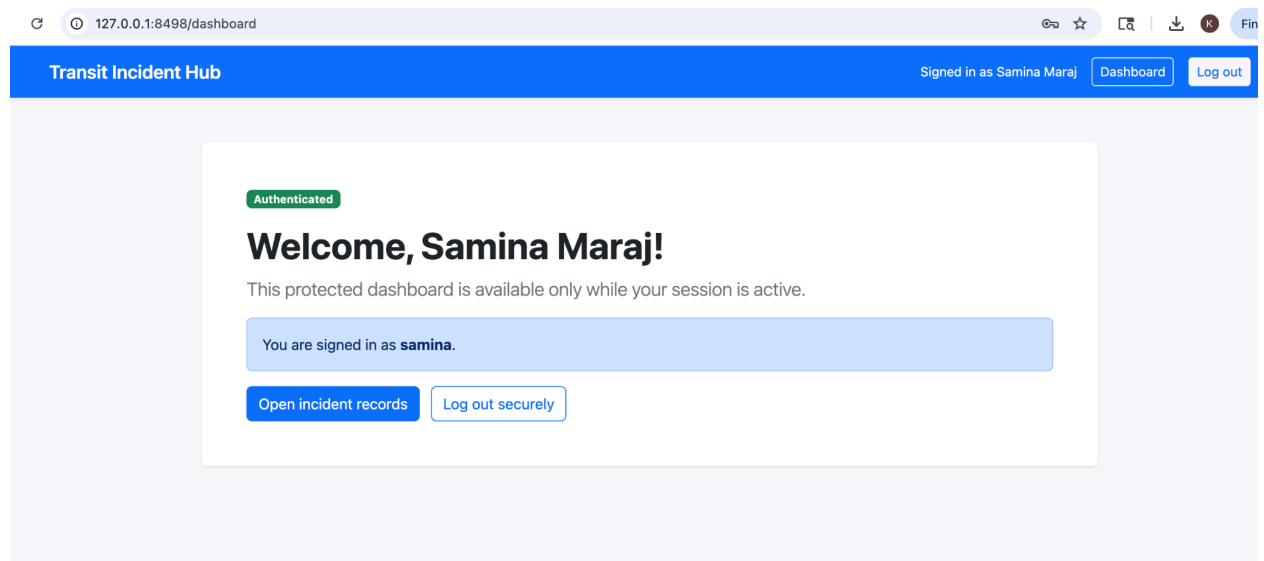


Figure 3. Protected dashboard after successful login.

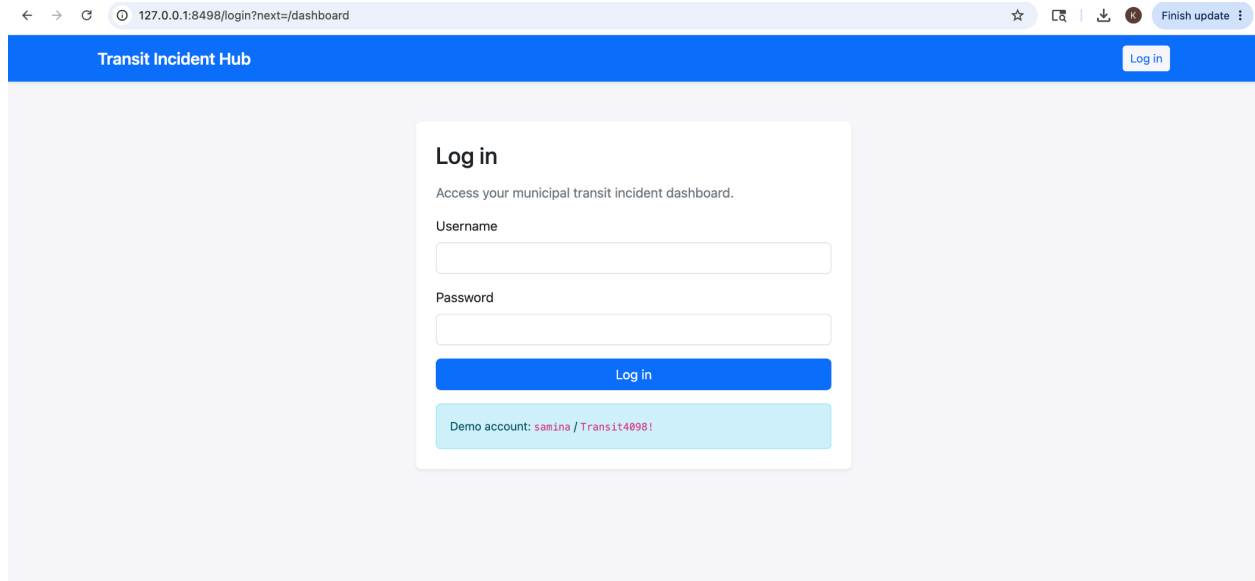


Figure 4. Unauthenticated access redirects to the login page.

Project template files

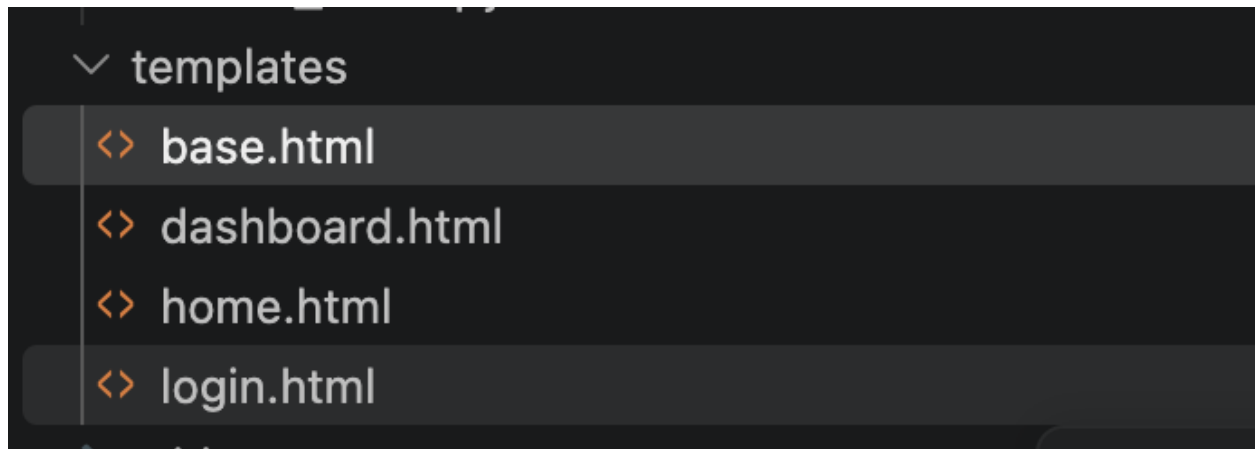


Figure 5. HTML templates used for the authentication pages.

```
reports > hw03 > {} verification.json > {} checks > {} authentication > [ ] cookie_flags
1  {
2    "timestamp_utc": "2026-09-18T03:57:39.041478+00:00",
3    "status": "PASS",
4    "commit": "39c4aeb017f82d99b2caedfba47e3f3336ab5071",
5    "checks": {
6      "required_files": {
7        "file_count": 5,
8        "missing": []
9      },
10     "authentication": {
11       "public_pages": {
12         "home": 200,
13         "login": 200,
14         "incidents": 200
15       },
16       "protected_before_login": 303,
17       "invalid_login": 401,
18       "successful_login": 303,
19       "dashboard_after_login": 200,
20       "cookie_flags": [
21         "secure",
22         "httponly",
23         "samesite=lax"
24       ],
25       "logout_dashboard": 303,
26       "expired_dashboard": 303
27     }
28   }
29 }
30
```

Figure 6. Automated verification of session cookie security flags.

The automated verification confirmed that the session cookie included the Secure, HttpOnly, and SameSite=Lax security attributes.

```

reports > hw03 > raw > {} retrieval_summaries.json > ...
1  {
2    "generated_at": "2026-09-18T03:48:48.742749+00:00",
3    "model": "sentence-transformers/all-MiniLM-L6-v2",
4    "summaries": [
5      {
6        "question_id": "q1",
7        "question": "What does the FRA Regulated Mode Major Security Events dataset contain?",
8        "technique": "Token",
9        "query_shape": [
10         384
11       ],

```

Figure 7. Embedding model and query vector shape recorded in retrieval_summaries.json.

```

reports > hw03 > raw > {} retrieval_summaries.json > [] summaries > {} 1
4    "summaries": [
36      {
43        "query_first_8": [
49         0.0012573173003212,
50         0.001650002901442349,
51         0.00035360537003725767
52       ],
53        "doc_vector_shape": [
54         5,
55         384
56       ],
57        "chunk_count": 32,
58        "avg_chunk_length": 5727.375,
59        "retrieval_latency_ms": 5.283790873363614,
60        "top1_cosine": 0.2772023379802704,
61        "mean_at_k_cosine": 0.32480737268924714,
62        "recall_at_k": 1,
63        "expected_source_file": "fra_regulated_mode_major_security_events.csv",
64        "answer_keyword_hit": true,
65        "source_file_hit": true
66      },
67      {
68        "question_id": "q1",
69        "question": "What does the FRA Regulated Mode Major Security Events dataset contain?",
70        "technique": "Sentence-window",
71        "query_shape": [
72         384

```

Figure 8. Retrieved document vector shape recorded in retrieval_summaries.json.

```

4119bab..d8b8f8d main -> main
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % unset SESSION_HTTPS_ONLY

python - <<'PY'
from fastapi.testclient import TestClient
from src.api import app

client = TestClient(app, base_url="https://testserver")
response = client.post(
    "/login",
    data={"username": "samina", "password": "Transit4098!"},
    follow_redirects=False,
)
print("Set-Cookie:", response.headers.get("set-cookie"))
PY
/Users/samina/Documents/data260-4098/.venv/lib/python3.12/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
from starlette.testclient import TestClient as TestClient # noqa
Set-Cookie: session=eyJlc2VyJjogeyJlc2VybmFtZSI6IChzYW1pbmE1LCAibmFtZSI6IChJTWVpbmEgTWYyYWo1fSwgImxvZ2luX3RpbmUjOiAxeGZnZSwwZmZlJmZpZzN3bmEuaqzRjQ.Q5iCpo5Cvu01hynj_ALNXca010g; path=/; Max-Age=900; httponly; samesite=lax; secure
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %

```

Figure 9. Raw Set-Cookie response header showing Secure, HttpOnly, and SameSite=Lax.

```

4119bab.d0b8f0d main -> main
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % unset SESSION_HTTPS_ONLY

python - <<'PY'
from fastapi.testclient import TestClient
from src.api import app

client = TestClient(app, base_url="https://testserver")
response = client.post(
    "/login",
    data={"username": "samina", "password": "Transit4098!"},
    follow_redirects=False,
)
print("Set-Cookie:", response.headers.get("set-cookie"))
|PY
/Users/samina/Documents/data260-4098/.venv/lib/python3.12/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'htpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
from starlette.testclient import TestClient as TestClient # noqa
Set-Cookie: session=eyJic2VyJjogeyJlc2VybmFtZSI6ICJzYWdpbmE1LCAibmFtZSI6ICJTYWlpmEgTWfYm9lfSgImxvZ21uX3RpbmU1O1AxNzg5NzEwMzIjYmMzgZn30=.aqzRjQ.05iCp05CvuDjhynj_ALNka010g; path=/; Max-Age=900; httponly; samesite=; secure
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % cd ~/Documents/data260-4098
source .venv/bin/activate

python - <<'PY'
import json

print("q1 - Token retrieval results")
print("rank | store_score | cosine_sim | chunk_len | preview")

with open("reports/rw03/raw/retrieval_outputs.jsonl") as f:
    for line in f:
        row = json.loads(line)
        if row["question_id"] == "q1" and row["technique"] == "Token":
            print(
                f'{"rank"} | '
                f'{"store_score":.4f} | '
                f'{"cosine_sim":.4f} | '
                f'{"chunk_len"} | '
                f'{"preview"}[1:80]'
            )
|PY
q1 - Token retrieval results
rank | store_score | cosine_sim | chunk_len | preview
1 | 0.5863 | 0.6264 | 1233 | events where an individual was immediately transported away from the scene for m
2 | 0.5495 | 0.5349 | 1395 | category such as event type and person type. FTA makes adjustments within this f
3 | 0.5206 | 0.5137 | 1263 | Transit Database (NTD) Safety and Security Event Data This page describes data c
4 | 0.5211 | 0.3785 | 236 | "ntd_id", "agency", "incident_description", "mode", "type_of_service", "trail_bus_ferr
5 | 0.4963 | 0.3890 | 236 | ction", "service_stop_control_device", "intentional_y_n", "vehicle_action", "action
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %

```

Figure 10. Per-query retrieval results for q1 using the Token chunking method. The table shows the rank, vector-store score, cosine similarity, chunk length, and preview for the five retrieved chunks.

```

templates > login.html > div.row.justify-content-center > div.col-md-6.col-lg-5 > div.card.border-0.shadow-sm
1  {% extends "base.html" %}
2  {% block title %}Login | Transit Incident Hub{% endblock %}
3  {% block content %}
4  <div class="row justify-content-center">
5      <div class="col-md-6 col-lg-5">
6          <div class="card border-0 shadow-sm">
7              <div class="card-body p-4">
8                  <h1 class="h3 mb-3">Log in</h1>
9                  <p class="text-secondary">Access your municipal transit incident dashboard.</p>
10                 {% if error %}<div class="alert alert-danger" role="alert">{{ error }}</div>{% endif %}
11                 <form method="post" action="/login">
12                     <div class="mb-3">
13                         <label class="form-label" for="username">Username</label>
14                         <input class="form-control" id="username" name="username" value="{{ attempted_username|default('') }}" required=
15                     </div>
16                     <div class="mb-3">
17                         <label class="form-label" for="password">Password</label>
18                         <input class="form-control" id="password" name="password" type="password" required>
19                     </div>
20                     <button class="btn btn-primary w-100" type="submit">Log in</button>
21                 </form>
22                 <div class="alert alert-info mt-3 mb-0 small">Demo account: <code>samina</code> / <code>Transit4098!</code></div>
23             </div>
24         </div>
25     </div>
26 </div>
27 {% endblock %}
28

```

Figure 11. Bootstrap classes used in the login template for the card, alerts, and login button.

Important authentication code

The following small section from `src/auth.py` checks whether the submitted username and password match a demo account:

```
user = USERS.get(username)
if user is None or user["password"] != password:
    return HTMLResponse("Invalid username or password", status_code=401)
```

The following section from `src/api.py` creates the session middleware. It allows FastAPI to remember the logged-in user between requests:

```
app.add_middleware(
    SessionMiddleware,
    secret_key=SESSION_SECRET,
    max_age=900,
    same_site="lax",
    https_only=SESSION_HTTPS_ONLY,
)
```

I used `SESSION_HTTPS_ONLY=0` only for local HTTP browser testing. The normal default keeps the secure cookie setting enabled.

Authentication verification

I ran `code/verify_hw03.py` to test the main authentication behaviors automatically. The verification passed. The public pages returned HTTP 200, the protected page redirected before login, the invalid login returned HTTP 401, and the valid login opened the dashboard. Logout and expired-session behavior also passed.

Test	Result
Public pages	PASS - HTTP 200
Dashboard before login	PASS - HTTP 303
Invalid login	PASS - HTTP 401
Successful login	PASS - dashboard opened
Cookie flags	PASS - secure, httponly, samesite=lax
Logout and timeout	PASS

2 Corpus and retrieval experiment

Corpus

For the retrieval part, I used two public Federal Transit Administration sources about transit safety and security. One source is the FRA Regulated Mode Major Security Events dataset. The other source is the FTA page explaining National Transit Database safety and security event data. Together, the local files are about 254 KB, which is above the required 200 KB minimum.

- fra_regulated_mode_major_security_events.csv
- ntd_safety_security_overview.html
- The source URLs, file sizes, access date, and SHA-256 hashes are recorded in CORPUS_MANIFEST.json.

Questions and embeddings

I wrote five questions in questions.yaml before running the retrieval experiment. The program converts the questions and document chunks into numerical vectors using sentence-transformers/all-MiniLM-L6-v2. Each vector has 384 numbers. Similar questions and passages should have more similar vectors.

Item	Value
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Vector dimension	384
Query vector shape	[384]
Retrieved document vectors shape	[5, 384]

Each question was converted into a vector with 384 values. The retrieved document shape [5, 384] means that the system returned five document chunks, and each chunk was represented by a 384-value vector.

Three chunking methods

A chunk is a smaller piece of a document. The retrieval system searches chunks instead of searching the whole document at once. I compared three methods:

- Token splitter: divides the text into regular pieces with a small overlap between pieces.
- Semantic splitter: tries to keep sentences with similar meaning together, so it creates fewer and larger pieces.

- Sentence-window splitter: retrieves a sentence together with nearby sentences to give it more context.

The main comparison code is in `code/run_retrieval_comparison.py`. The important setup looks like this:

```
"Token": TokenTextSplitter(chunk_size=256, chunk_overlap=32),
"Semantic": SemanticSplitterNodeParser.from_defaults(...),
"Sentence-window": SentenceWindowNodeParser.from_defaults(window_size=2, ...)
```

For every question, the program saved the five highest-scoring chunks and recorded the source file, score, cosine similarity, length, preview, and retrieval time.

3 Results

Method	Chunks	Avg length	Top-1	Mean@5	Recall@5	Latency
Token	334	653.34	0.6083	0.4983	5/5	32.40 ms
Semantic	32	5727.38	0.5151	0.3471	5/5	21.85 ms
Sentence-window	579	316.54	0.5622	0.4492	4/5	28.73 ms

The program saved 15 summaries because there were five questions and three methods. It also saved 75 retrieval rows because it kept the top five results for each of those 15 tests.

In this experiment, Token chunking performed best overall. It had the highest average top-1 similarity and mean similarity@5, and it found the expected answer for all five questions. Semantic chunking was the fastest and created the fewest chunks, but its chunks were very large. Sentence-window chunking created the most chunks and performed well on some overview questions.

Misleading retrieval results

Three top-ranked results were marked as potentially misleading. This means the result with the highest similarity score did not contain one of the expected answer keywords. It does not mean that the entire search failed. It shows that a passage can sound related to a question without directly answering it. This is why a real system should check the retrieved text and source instead of trusting only the highest score.

The raw results are saved in:

- reports/hw03/raw/retrieval_summaries.json
- reports/hw03/raw/retrieval_outputs.jsonl
- reports/hw03/raw/misleading_candidates.json

4 Conclusion

This homework helped me understand two parts of the application. First, authentication protects the dashboard while keeping public incident pages available. Second, the way documents are divided can affect retrieval quality and speed. For this corpus and these five questions, TokenTextSplitter was the best overall method. However, the experiment also showed that a similarity score is not a guarantee that the retrieved passage is the correct answer.

The final project files include the source code, screenshots, corpus manifest, questions, raw retrieval results, metrics, run log, and verification record.

Files I used

- src/auth.py and src/api.py for authentication and sessions
- templates/ for the web pages
- code/verify_hw03.py for automated authentication testing
- code/run_retrieval_comparison.py for the RAG comparison
- reports/hw03/METRICS.md and RUN_LOG.txt for recorded results

AI USE

I used AI to explain FastAPI sessions, LlamaIndex chunking, and debugging errors. I wrote and ran the commands, checked the screenshots, and reviewed the results myself.

I independently verified the cookie settings instead of trusting the AI explanation. The local browser used HTTP, so it was not suitable for proving the Secure cookie flag.

I detected this by checking the environment setting and running code/verify_hw03.py with an HTTPS test client.

I kept local HTTP mode only for browser screenshots and used the automated HTTPS verification for the final cookie evidence. The verification confirmed Secure, HttpOnly, and SameSite=Lax.

Repository: data260-4098

GitHub: <https://github.com/SaminaMaraj/data260-4098>

Final tag: hw3

Domain: Municipal transit incidents

SID4: 4098

Port: 8498